hw1

October 15, 2026

1a. Even money bets at market probability 0.6 means that for every \$1 I bet I get \$2/3 profit (plus my initial \$1 back) if I win. If my Utility is $d + X$ where X is my net payoff from my bet, and I bet a then

$$\mathbb{E}[d + X] = d + \mathbb{E}[X] = d + (\frac{2}{3}(\frac{17}{20})a - \frac{3}{20}a) = d + \frac{5}{12}a$$

\ Since the coefficient 5/12 is positive, expected utility is increasing in a, so the value for a that maximizes this, where a ranges from 0 to d, is d itself, so Victor should bet all his money. The linear utility on money is not always good, because even if on average you make money, losing everything sometimes is way worse than gaining money most of the time.

b) With the log utility function, our expectation is now given by :

$$\frac{17}{20}\log(d + \frac{2}{3}a) + \frac{3}{20}\log(d - a)$$

Setting the derivative equal to 0 and solving for a we get

$$(\frac{17}{20})(\frac{2}{3})\frac{1}{d + 2/3a} - (\frac{3}{20})\frac{1}{d - a} = 0 \implies a = \frac{5}{8}d$$

c) If I had a net worth of \$500,000 I would not make the bet. The bet is too high variance, losing 5/8 of all of my money is a big loss and I would rather make smaller variance gambles that are slightly less positive EV than a huge variance but higher EV bet.

2) The cross entropy loss is given by:

$$L(Y, f(X)) = -\left(Y \log f(X) + (1 - Y)\log(1 - f(X))\right)$$

We then have

$$\arg\min_f \mathbb{E}_{X,Y}[L(Y, f(X))]$$

By iterated expectations, we have

$$\mathbb{E}_{X,Y}[L(Y, f(X))] = \mathbb{E}_X\left[\mathbb{E}_{Y|X}[L(Y, f(X))]\right]$$

We have

$$\mathbb{E}_{Y|X=x}[L(Y, f(X))] = -\Pr(Y = 1|X = x)\log(f(x)) - \Pr(Y = 0|X = x)\log(1 - f(x))$$

To minimize the expectation of the conditional, since f is unrestricted, we can take $f(x) = t$ to be the value t which minimizes the conditional pointwise. We have

$$g(t) = -\Pr(Y = 1|X = x)\log(t) - \Pr(Y = 0|X = x)\log(1 - t)$$

then

$$g'(t) = \frac{-\Pr(Y = 1|X = x)}{t} + \frac{\Pr(Y = 0|X = x)}{1 - t} = \frac{-\Pr(Y = 1|X = x)}{t} + \frac{1 - \Pr(Y = 1|X = x)}{1 - t}$$

Setting to 0 and solving, we get

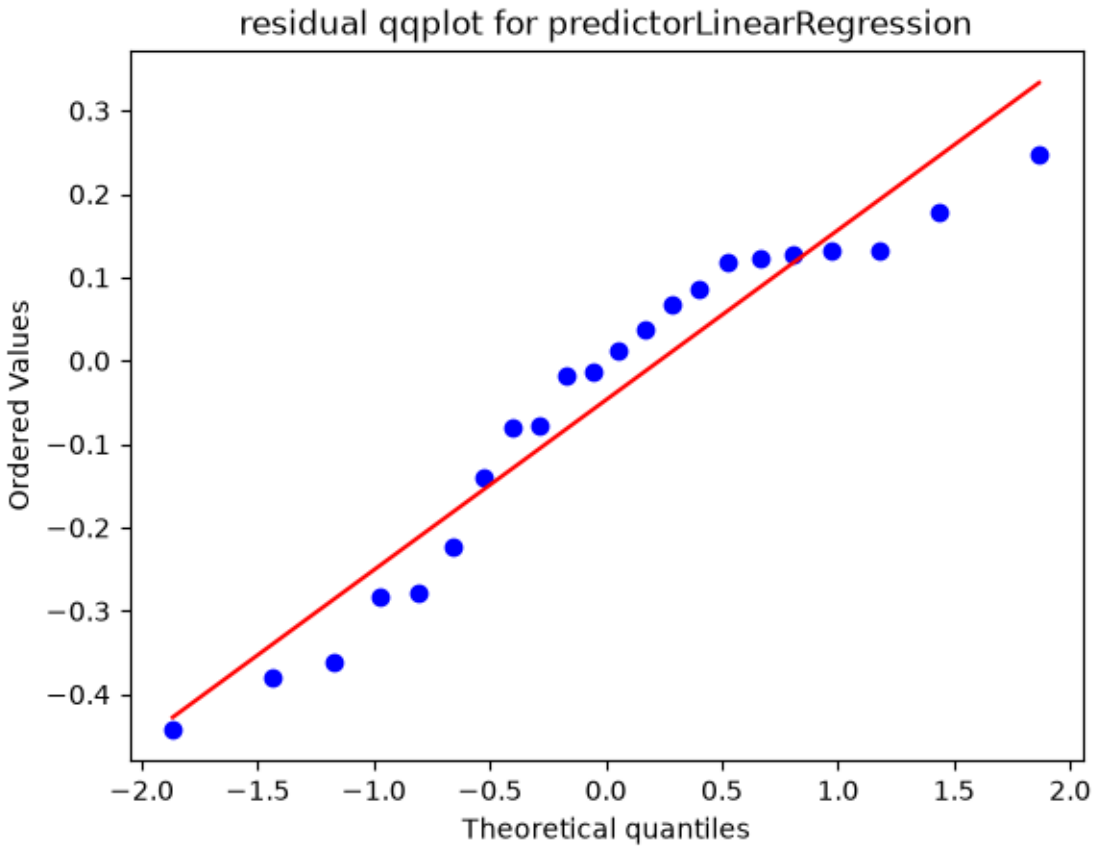
$$t = \Pr(Y = 1|X = x)$$

Since $g'(t)$ is negative when $t < \Pr(Y = 1|X = x)$ and positive when $t > \Pr(Y = 1|X = x)$, g attains a minimum at this point. Minimizing the conditional expectation pointwise minimizes the expectation of the conditional, so we have $f^*(x) = \Pr(Y = 1|X = x)$ and the claim follows.

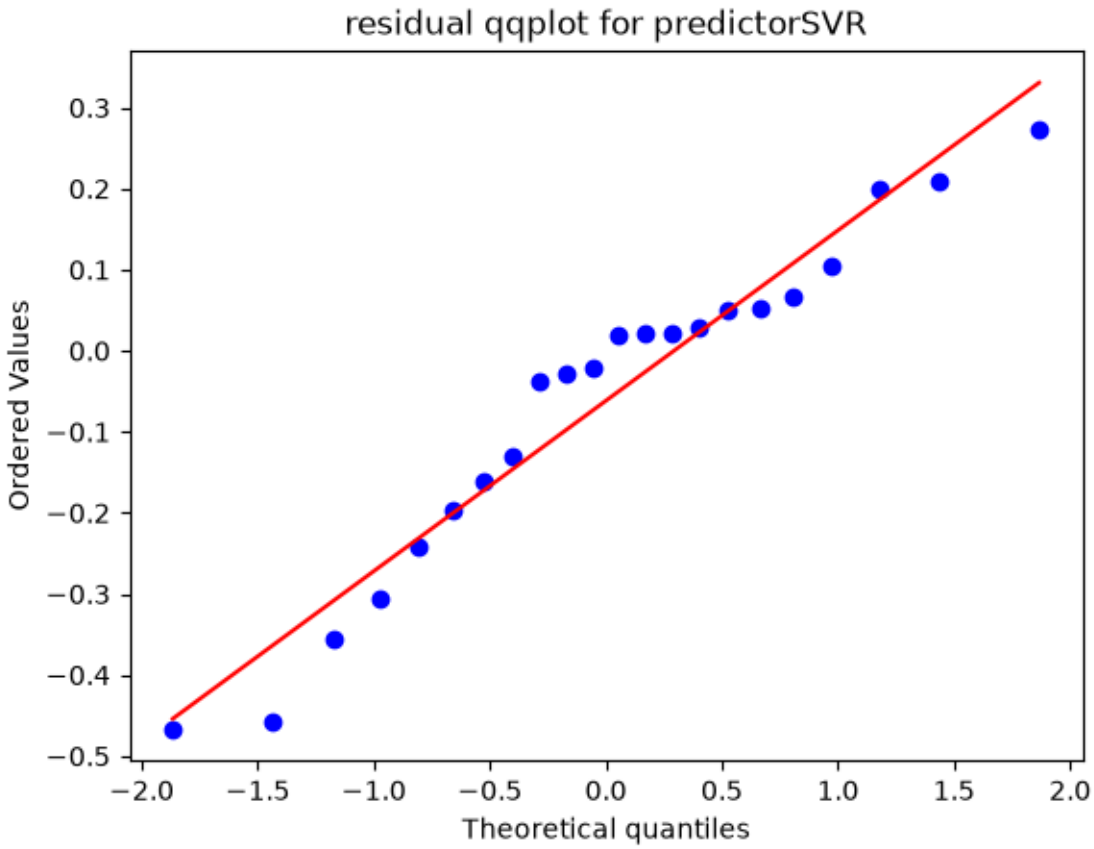
3) In practice, we don't necessarily need arbitrary precision, once we get to good enough, we are good enough. Plus, with finite sampling there's no guarantee we'll get arbitrarily close anyway. So in practice we settle with likely getting to 'close enough' faster with a constant learning rate rather than guaranteeing arbitrary precision but slower initial steps.

[1]:
```
#Problem 4
import importlib
import prob4
importlib.reload(prob4)
from prob4 import regress_and_predict, problem4

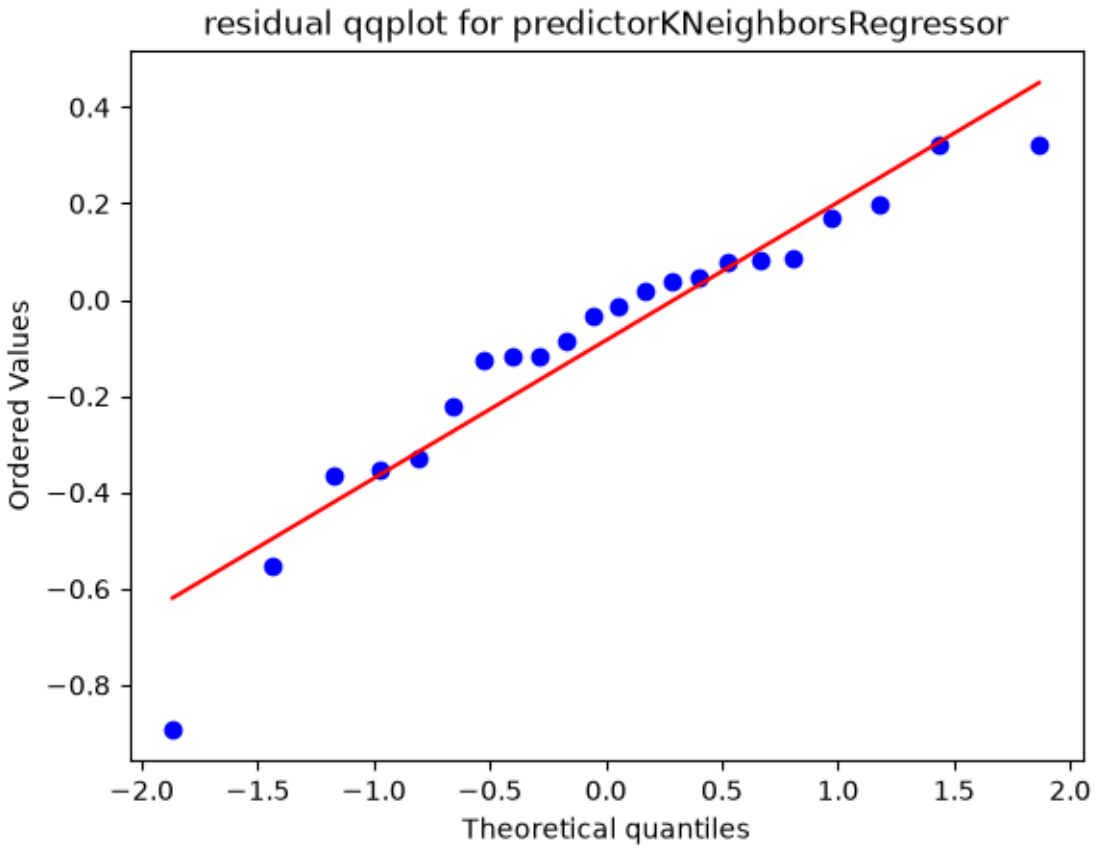
problem4()
```




```
statistic = 2.5472, pval = 0.2798
```




```
statistic = 1.3817, pval = 0.5012
```




```
statistic = 7.6526, pval = 0.02179
```

SVR and linear regression residuals suggest normal assumption with higher pvalues and points near the line on QQ plots. KNN look less normal.

[12]:
```
#problem 5a
import prob5
importlib.reload(prob5)
prob5.prob5a_logistic_class()
```

```
baseline acc: 0.9649122807017544
jax acc: 0.9649122953414917
```

the accuracy of the logistic is very similar to the baseline.

[13]:
```
prob5.prob5b_probit_class()
```

```
baseline acc: 0.9649122807017544
jax acc: 0.9649122953414917
```

the accuracy of the probit classifier is also very similar to the baseline

[14]:
```
#prob 5a regression
prob5.prob5a_linear_reg()
```

```
linear regression error: 0.19207172045887988
jax MAE: 0.19207163
avg test loss: 0.051611334
```

This model and the baseline perform very similarly

[15]:
```
prob5.prob5b_t_reg()
```

```
linear regression error: 0.19207172045887988
jax MAE: 0.18862274
avg test loss: 0.9985551
```

The baseline and the model with the t distribution loss, although it is less similar than the previous (it does slightly better)

[ ]:
```

```